

E-Voting via Upgradable Smart Contracts on Blockchain

Mohammad Saim

Department of Computer Engineering
Zakir Husain College of Engineering and Technology
Aligarh Muslim University, Aligarh, India
mohammadsaim@zhcet.ac.in

Ilma Shah

Department of Computer Engineering
Zakir Husain College of Engineering and Technology
Aligarh Muslim University, Aligarh, India
ilmashah0703@gmail.com

Mohd Mamoon

Department of Computer Engineering
Zakir Husain College of Engineering and Technology
Aligarh Muslim University, Aligarh, India
mohdmamoon@zhcet.ac.in

Abdus Samad

Department of Computer Engineering
Zakir Husain College of Engineering and Technology
Aligarh Muslim University, Aligarh, India
abdussamad@zhcet.ac.in

Abstract—The proposed work is a blockchain-based E-Voting system which is designed to incorporate privacy, authenticity, and verifiability in the electoral process. Solidity programming language is used to implement smart contracts stored on the Ethereum blockchain which are executed automatically when predefined conditions are met. An Ethereum-based E-Voting decentralised application is deployed which authenticates users with the help of biometric factors and verifies it with the existing E-Aadhaar database.

In this model, the paper presents the novel idea of deploying an upgradable smart contract where the election manager can decide whether the voting application would require either a version change, virtual upgrades or feature modifications in the future. This is achieved by using the OpenZeppelin plugin deployed on truffle which uses the concept of “proxy pattern.” This methodology helps us to improve and enhance the logic of smart contracts that are normally non-editable once deployed. The parameters used for authentication include the user’s Email-ID, Registration ID(Aadhar), and facial scan for biometric verification along with other details related to the voter’s biodata.

Index Terms—E-Voting, Ethereum, Solidity, Upgradable Smart Contract, Blockchain, OpenZeppelin

I. INTRODUCTION

Electronic Voting is seen as a tool for improving the efficiency of an electoral process. Effective implementation of E-voting has been seen in various countries around the globe. An electoral system, which strives to elect office bearers, leaders, and people who will hold important positions in a society or organisation, is critical to the efficient operation of organisations such as NGOs, corporate houses, local communities, and government agencies.

The use of ballot paper is a traditional method of voting, which provides a “trust factor” because it is very difficult to hijack them electronically. Moreover, no complex guide is

required for people to understand this age-old voting process. It is not, however, an ideal method of holding elections due to its high risk of voter fraud and lack of potential for automation. It is not economically viable, and fully equitable and counting ballots is a time-consuming operation. The most serious problem is the insecure and costly transportation of voting papers [1].

Analysis showed that an E-Voting system is faster, economical and less prone to human errors. It improves voter convenience, increases voter privacy and is seen to have more accurate results [2].

A big drawback the present E-Voting system carries is that it is centralised and regulated by a single authority. A centralised system has the liability of a single point of failure, hence, it is unsafe. This reduces transparency (voters are sceptical of the fact that their vote has been counted) and is one of the factors in low voter turnout in elections. Hence, it is essential for the process to satisfy the conditions, such as eligibility, un-reusability, transparency, privacy, and fairness [3].

An alternative is e-Voting via Blockchain.

Blockchain can be defined as a decentralised ledger that allows authorities, who do not trust each other to cooperate, coordinate, and collaborate in a decision-making process. The data on a blockchain cannot be modified, which makes it a viable alternative to the existing voting system [4].

II. BLOCKCHAIN TECHNOLOGY

A. Blockchain Characteristics

1. Cryptographic security: Blockchain’s primary security method is cryptography. It enables the storage and exchange of data in a format that is inaccessible to third parties, and only the individual with the ability to decrypt the contents of the text may receive and decode the message delivered to them.

2. Decentralisation: Decentralisation enables blockchain technology to eliminate the authority of any central party from having access to all the data.

3. Anonymity and Transparency: Digital signature in blockchain protects a user's online identity and secures its integrity and reliability. The blockchain stores documents encrypted in digital code in transparent, database systems that display robustness against tampering, modification, and destruction of the records [5].

B. Preliminaries

1. Ganache: An Ethereum simulator that enables the development, deployment, and testing of smart contracts in a reliable environment [6]. In our implementation, Ganache UI (a desktop application supporting Ethereum) is used. On creation, the workspace overview provides a fixed number of accounts containing 100 test ethers in each account.

2. Metamask: A cryptocurrency wallet used for interaction with the Ethereum network [7]. In our model, Metamask is linked with Ganache which imports accounts for test transactions on a selected custom RPC.

3. Truffle: A truffle box used for initializing smart contracts with a react framework is used to act as assisting boilerplate for our application.

4. The Use of an Authorized Identity Card Provided by the Government of a Country: In our implementation, an Aadhar card acts as a special identification card. It is issued by the government of India and uploaded by the user which is sent to the database for verification during registration. It's a 12-digit unique identity number that can be obtained voluntarily by the citizens of India. The data is collected by a regulatory authority with the registered database of voters.

For elections conducted for residents of India, the model takes the help of the Virtual ID (VID) issued by the Unique Identification Authority of India UIDAI, a national agency storing biometric and demographic information in a database. The VID is used for user authentication and, once generated, stays valid for a minimum of 24 hours until the user generates it again.

C. Proposed Solution

The requirements of an E-Voting system overlap with the characteristics of blockchain, i.e., immutable, decentralized, anonymous, cryptographically secure, and cost-effective. Blockchain is an effective model for voting because every transaction is validated by all the nodes in the network to ensure that there is no node as an entity or a person that has more authority than others. It acts as a digital database containing information that can be accessed within a large decentralized, publicly accessible network. This amounts to an increase in trust and transparency among the voters.

III. RELATED WORKS

Blockchain-based E-Voting systems have been used in some parts of the world [8]. Zug, Switzerland conducted a

blockchain-based e-vote and 79 percent of residents approved it. Over 220 people in Zug have a registered digital ID and were eligible to vote on the blockchain-based platform. Blockchain-based voting was found quicker and easier when compared to the traditional ballot system.

In Tsukuba, Japan, the voters were issued a 12-digit social security number for authentication purposes. The voting conducted on Blockchain was found to be convenient.

Moscow's Active Citizenship program started using blockchain for voting in December 2017. The popular polls saw 137,000 to 220,000 participants and the platform witnessed up to 1000 transactions per minute.

Agora, an end-to-end verifiable blockchain-based voting ecosystem, tallied election results in two districts of Sierra Leone in 2018. 4,00,000 ballots were entered into Agora's blockchain ecosystem. This helped avoid electoral fraud. This was only a partial deployment as the election results were only corroborated by blockchain.

There have been various successful executions of blockchain-based E-voting systems around the world. The most often cited research developed an E-Voting system promoting transparency and security. Hardwick et al. have proposed an e-voting system that replaces the traditional ballot box with blockchain. The proposed system offers decentralisation and allows the voter to change his/her vote. However, in this system, the voter authentication is done by a central authority [9].

Koç et al. proposed a system by deploying smart contracts on the Ethereum blockchain. The system focuses on a transparent system that does not allow duplication of votes. The Ethereum blockchain will hold the record of ballots and votes. This paper does not address the issue of voter authentication [10]. In [11], an E-Voting application is developed based on blockchain that provides decentralisation and does not allow duplicate voting, making it a reliable application. In this system, voter registration is done in advance and voter authentication is conducted by a central authority.

In [12], several specific requirements, such as privacy, eligibility, convenience, receipt-freeness and verifiability, were taken into account when designing the system. The proposed system aims to achieve secure digital voting without compromising its effectiveness. A user-friendly web-based interface is developed which ensures voter anonymity.

Prabhu et al. proposed a system with a two-step authentication process of face recognition and an OTP system which provides added security [13].

Deepika et al. proposed that biometric identification can deliver more accurate findings while also being trustworthy. In this paper, the voter enters their fingerprint impression as an input and if the input is matched with the available record in the database, he/she is granted permission to vote [14].

IV. SYSTEM IMPLEMENTATION

In order to further strengthen voter confidentiality and integrity, and to ensure that the election system should not

enable fraud, curtailed or constrained voting, voters will have to vote in an administered ecosystem.

A. System Layout

The planned decentralized voting system mainly consists of two entities, the voters and the election manager. The voters must be registered nationals of the country and would also have to register themselves on the blockchain application. They must possess a wallet linked to Metamask, an Ethereum-based browser extension for successful transactions during the vote cast. The voter is entitled to vote only once in the election round. Once the vote is cast, the address of the voter is stored and added to the chain.

The administrator node is responsible for initiating the election process, authenticating the user through their existing database, declaring the timeline of the election and announcing results when the process comes to an end. However, it is important to note that the administration is not in power to meddle or interfere with the voting system or the smart contract after deployment. This guarantees the security and freedom of doubt about the credibility of the system for a voter.

In our work, we set up an upgradable smart contract, deployed in Solidity programming language to achieve this goal. OpenZeppelin, a software for managing and developing Ethereum projects is introduced with the “Upgrades Plugin” feature to enhance the longevity of the voting software [15]. The smart contract is first linked with Ganache. A client-side web application is then designed to display the interface to the user. Before voters can move towards the voting application, they are prompted to register themselves as a voter by defined inputs like name, contact, email address, etc.

B. Original Contribution

1) *Problem Statement:* Suppose an election software is used in an organisation to elect their project representatives at regular intervals. At first, there would be no need to alter the software. But as time passes, people may feel the need to add some features to the voting process or fix an anomaly or a bug.

For a contract that was deployed as non-upgradable, these are the steps the organisation can take:

1. The contract will be deployed in a new version from scratch.
2. All states need to be manually migrated to a new contract from an old one. This process may result in high amount of gas fees.
3. Using the new address in the updated contract.

These methods are practical but can consume a lot of gas fees, time and resources for the organizer/manager.

2) *Proposed Solution:* In our work, we take use of the existing technology of upgradable smart contracts and integrate it with our election smart contract. OpenZeppelin “Upgrades Plugin” can be used to upgrade the deployed smart contracts

to modify their code while preserving their address, state, and balance. This allows the manager to iteratively add new features to the project, or fix any bugs he/she may find in production. There exist three methods to upgrade a smart contract:

- Parametrize Method
- Migration Method
- The Proxy Method

In this paper, we use the Proxy method of upgradation for our smart contract. The Proxy method is chosen for its ability to update the state, keep the contract address intact and allow the manager to update any type of logic in our contract. Proxies are known to use low-level functionality, for example, the `delegatecall` method. This approach is similar to a message call with the feature that the code present at the target address is executed when the contract is invoked, with `msg.sender` and `msg.value` retaining their initial values.

The proxy contract, at all times, contains the same address and whenever a new implementation contract is created, the proxy routes or points toward the correct implementation contract that holds the updated logic.

Below are the contracts and terms used in our application:

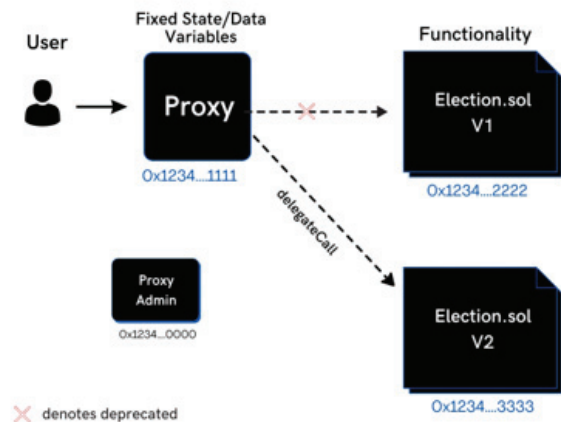


Fig. 1. Proxy contract calling the upgraded version of Election contract

- **Election.sol:** The contract contains all the protocol codes which initialize the first implementation of our Election smart contract. It essentially contains a structure comprising the name, identity and vote count. An `addCandidate()` function is responsible for adding Candidates and a `vote()` function checks the eligibility of both the voter and the candidate.

- **Proxy Contract:** The contract responsible for routing or pointing towards the correct implementation contract. Hence, a voter calling a function will be pointed to the function in the new implementation contract. An added feature of the proxy contract is that all our storage variables and states will be stored in the proxy itself.

- **Voters:** They are responsible for making calls to the proxy contract.

```
function upgrade(address newElection) external {
  if (msg.sender != admin) fallback();
  implementation = newElection;
}
```

Fig. 2. Code block for verifying admin for Proxy

```
const WebcamCapture = () => {

  const webcamRef = React.useRef(null);
  const videoConstraints = { //declare frame dimensions
    width:200,
    height:200,
    facing mode: 'user'
  }
  const [name, setName] = useState('')

  const function capture {
    imageSrc=webcamRef.current.getScreenshot()
    axios post to ('http://127.0.0.1:5000/api', {data: imageSrc})
    .then( Response => {
      setName(response.data) //promise based HTTP client
    })
    .catch(error => { console.log( $error) //log in case of error
    })
  }
  [webcamRef]
};
```

Fig. 3. Pseudocode snippet for webcam capture file in Javascript

- **Proxy Admin:** The authority that is in charge of upgrading to the new implementation contract. Only one ProxyAdmin per network is deployed. When we launch our system live on the web, the ProxyAdmin is owned by the deployer address. The code block shown in figure 2 verifies the administrator for the Proxy contract.

3) **Methodology:** For our objective of deploying upgradable smart contracts, we consider the Transparent Proxy method for upgrading contracts which makes sure that the voters can only call functions of the implementation contracts. On similar grounds, election managers can only call admin functions. However, the implemented logic resides in the proxy contract, making it costlier for deployment.

The Truffle Upgrades plugin provides a `deployProxy` function to deploy our upgradable contract. This automatically checks that the Election.sol contract is safe for upgradation, sets up a proxy admin, deploys the implementation contract for the Election.sol contract and creates a proxy for initialization. For future migration, we apply the `upgradeProxy` function to upgrade the deployed instance to a new version. Referring to Fig 1, the proxy points to the updated ElectionV2.sol contract or the existing Election contract which can be modified and recompiled. The plug-in notes the change automatically.

Below are defined the essential files and components made for both the front-end and the back-end which are implemented for the system.

1. **App.js:** This file is responsible for gathering all components of the application's interface. It is further used to access the web3 library along with loading the blockchain data onto the system.

2. **getWeb3.js:** The web3.js library of the Ethereum blockchain is a collection of libraries which allows interaction with a local/remote Ethereum node. The web3 JavaScript library interacts with the Ethereum blockchain. It sends transactions, interacts with smart contracts, and retrieves user accounts for accessibility if needed.

3. **Signup.js and Login.js:** The Signup file is responsible for the initial registration of the user with basic details like name, email, password and contact along with the Special Identification Card(SID). In our case, uploading of Aadhar card for Indian users is prompted. Login.js contains operations of generation of One-Time-Password along with data submission and verification.

4. **Body.js:** This contains the main user interface and home screen of the application which contains the given list of candidates along with a vote button linked to them.

5. **Utils Folder:** It contains files like "webcam.js", and "timeout.js" which are employed on both the registration and home screen. The former is used to capture and/or store the image of the voter. The latter is used to put a timeout session on the home screen which logs out the user in case he's inactive for a given amount of time.

6. **Election.sol:** It contains the smart contract responsible for the addition/removal of candidates along with checks and bounds on the voter like the ability to vote only once. It further checks the validity of candidates, records that the voter has cast his vote, and updates the candidate vote count.

Figure 4 gives a walkthrough process of the entire application in a stepwise manner.

The process is broken down into six different steps starting from voter registration up to the time of vote tallying.

1. **Voter Registration** An eligible voter registers himself/herself on the application by filling out parameters like name, email, and contact. The user is prompted to upload his/her Aadhar card, which will be used to verify the identity of the user. A username and password are also generated by the user which will be used for logging in to the system.

2. **Voter Login** Once registered, the voter can log in to his unique address account by filling out credentials like username and password. To confirm the final steps of registration a one-time password (OTP) is generated and sent to the person's registered email address.

3. **Biometric Verification via Facial recognition using Flask** The Logging-In system is supported by a face detection model, realised by integrating a Python-based model to authenticate user login with the help of a Flask framework in conjunction with the ReactJs web application.

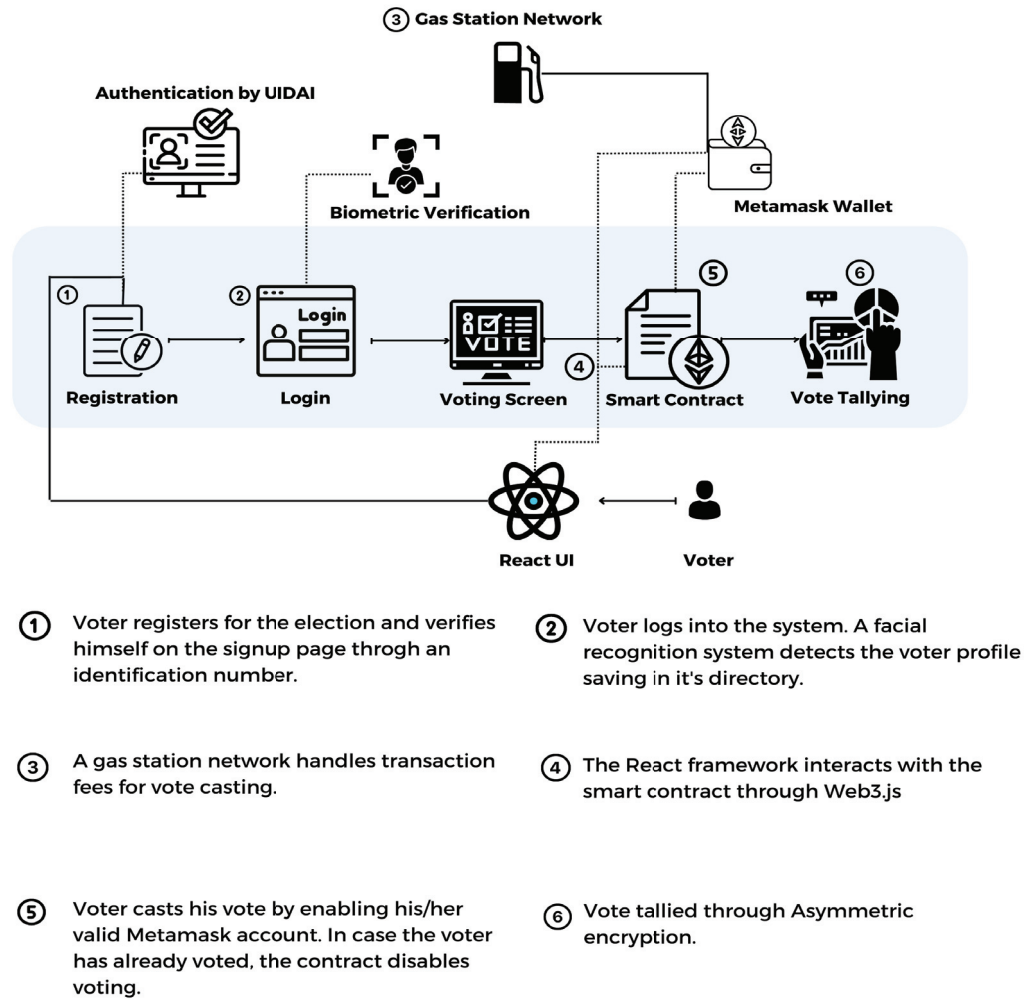


Fig. 4. Stepwise process of vote casting from the time of voter registration up to the time of vote being tallied.

```

fallback() external payable {
  require(msg.sender != admin);
  election.delegatecall.value(msg.value)(msg.data);
}

```

Fig. 5. Code block for calling the Election contract via delegate call by proxy

A face is detected during the login procedure and saved in a back-end directory. These are then matched with the already available Aadhar-UIDAI database (user database). For validation, Face-Id is captured every time the user logs in to the application. This is done to keep in check that only verified users sign in every time they go to the voting page. This method, however, carries some error margin due to bad image quality on the Aadhar card.

4. Handling of Transaction fees Gas is a unit of measurement for the amount of computing work necessary to carry out specified actions on the Ethereum Blockchain [16]. In the case of application, smart contracts containing different operations have a particular gas cost associated with them. This is charged to compensate the miners who have added the particular transaction to the block.

In order to ensure that the burden of voting costs does not fall upon potential voters, the election management will ensure that the minimum required gas fee is transferred to the voting account. For this purpose, the management sets up a gas station network(GSN) which is a decentralized network of relayers. It allows the user/voter to cast his/her vote so they do not need to hold Ether to pay for gas, easing their onboarding process where the cost is handled by the manager.

5. Vote Casting The vote is cast by the registered voter by selecting his/her candidate of choice. Once cast, the vote creates a block representing the vote transaction. Once the vote is verified by every node, the vote gets added to the blockchain.

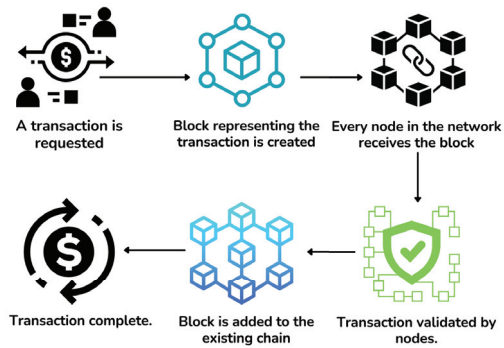


Fig. 6. A Transaction Execution on a Blockchain

The transaction process is handled by the Metamask wallet. Once the vote is submitted, a block representing that transaction is created and sent to every node for validation. This proceeds to the block being added to the existing blockchain, whose proof is supported by the increment in the candidate's vote count, as illustrated in Fig. 6. Code line shown in Fig. 5 restricts the transaction in case someone tries to vote on a second occasion.

At the time of voting the Ethereum virtual machine(EVM) checks for enough gas fee to execute the voting operation. In case there's not enough gas left, the whole transaction is reverted with the OUT-OF-GAS exception and all state changes are rolled back. However, the transaction fee of the miner is still deducted from the voting account to prevent Denial-Of-Service(DOS) attacks.

6. Vote Tallying Asymmetric encryption makes use of two distinct and linked keys. The public key is used to encrypt data, whereas the private key is used to decrypt it. Here, the public key is provided by the election management to the user.

For verifying the submitted vote, the election body uses the private key to decrypt the stored data of the registered voter and conclude the authenticity of the vote. The vote count keeps getting updated as voters cast their votes. For the sake of transparency, the current number of votes may also be displayed to the voter outside the voting environment.

V. CONCLUSION

The proposed E-Voting application based on the Ethereum blockchain overcomes the drawbacks of a centralised voting system. The application increases voter trust, system security, and cost-effectiveness. Biometric verification strengthens the authenticity of the system. The deployed upgradable smart contract allows the election manager to modify the code by making necessary changes for future elections. This methodology upgrades the non-editable functionality of smart contracts. However, the transactions per second for an Ethereum blockchain falls short for elections with a large population of voters. The upgradability of smart contracts faces the risk

of data separation and added complexity in the code. Thus, the issue of scalability and security improvisations is a future research topic in this area.

REFERENCES

- [1] Adewale, Olumide Boyinbode, Olutayo Adekunle, Salako. (2020). A Review of Electronic Voting Systems: Strategy for a Novel. *International Journal of Information Engineering and Electronic Business*. 12. 19-29. 10.5815/ijieeb.2020.01.03.
- [2] Dimitris A Gritzalis, Principles and requirements for a secure e-voting system, *Computers Security*, Volume 21, Issue 6, 2002
- [3] Bungale, Prashanth Sridhar, Swaroop. (2008). A Framework for Receipt Issuing, *Contendable Remote Poll-Site Voting*.
- [4] R. Shaikh, "Blockchain Based Cloud Storage of Patients Health Records," 2022 IEEE Delhi Section Conference (DELCON), 2022, pp. 1-5, doi: 10.1109/DELCON54057.2022.9753574.
- [5] Shubhani Aggarwal and Neeraj Kumar. Chapter seven - basics of blockchain. In Shubhani Aggarwal, Neeraj Kumar, and Pethuru Raj, editors, *The Blockchain Technology for Secure and Smart Applications across Industry Verticals*, volume 121 of *Advances in Computers*, pages 129–146. Elsevier, 2021.
- [6] <https://trufflesuite.com/ganache/>
- [7] <https://trufflesuite.com/docs/truffle/getting-started/truffle-with-metamask>
- [8] Nir Kshetri and Jeffrey Voas. Blockchain-enabled e-voting. *IEEE Software*, 35(4):95–99, 2018.
- [9] Freya Sheer Hardwick, Apostolos Gioulis, Raja Naeem Akram, and Konstantinos Markantonakis. E-voting with blockchain: An e-voting protocol with decentralisation and voter privacy. In 2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), pages 1561–1567, 2018.
- [10] Emre Yavuz, Ali Kaan Koc, Umur Can C abuk, and G okhan Dalkilic . Towards secure e-voting using ethereum blockchain. In 2018 6th International Symposium on Digital Forensic and Security (ISDFS), pages 1–7, 2018.
- [11] Ali Mansour Al-madani, Ashok T. Gaikwad, Vivek Mahale, and Zeyad A.T. Ahmed. Decentralized e-voting system based on smart contract by using blockchain technology. In 2020 International Conference on Smart Innovations in Design, Environment, Management, Planning and Computing (ICSIDEMPC), pages 176–180, 2020.
- [12] Kashif Mehboob, Junaid Arshad, and Muhammad Khan. Secure digital voting system based on blockchain technology. *International Journal of Electronic Government Research*, 14:53–62, 01 2018.
- [13] S Ganesh Prabhu, A Nizarahammed., S Prabu., S Raghu., R.R. Thirrunavukkarasu, and P. Jayarajan. Smart online voting system. In 2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS), volume 1, pages 632–634, 2021.
- [14] J. Deepika, S. Kalaiselvi, S. Mahalakshmi, and S. Agnes Shifani. Smart electronic voting system based on biometric identification-survey. In 2017 Third International Conference on Science Technology Engineering Management (ICONSTEM), pages 939–942, 2017.
- [15] <https://www.openzeppelin.com/>
- [16] <https://ethereum.org/en/developers/docs/gas/>